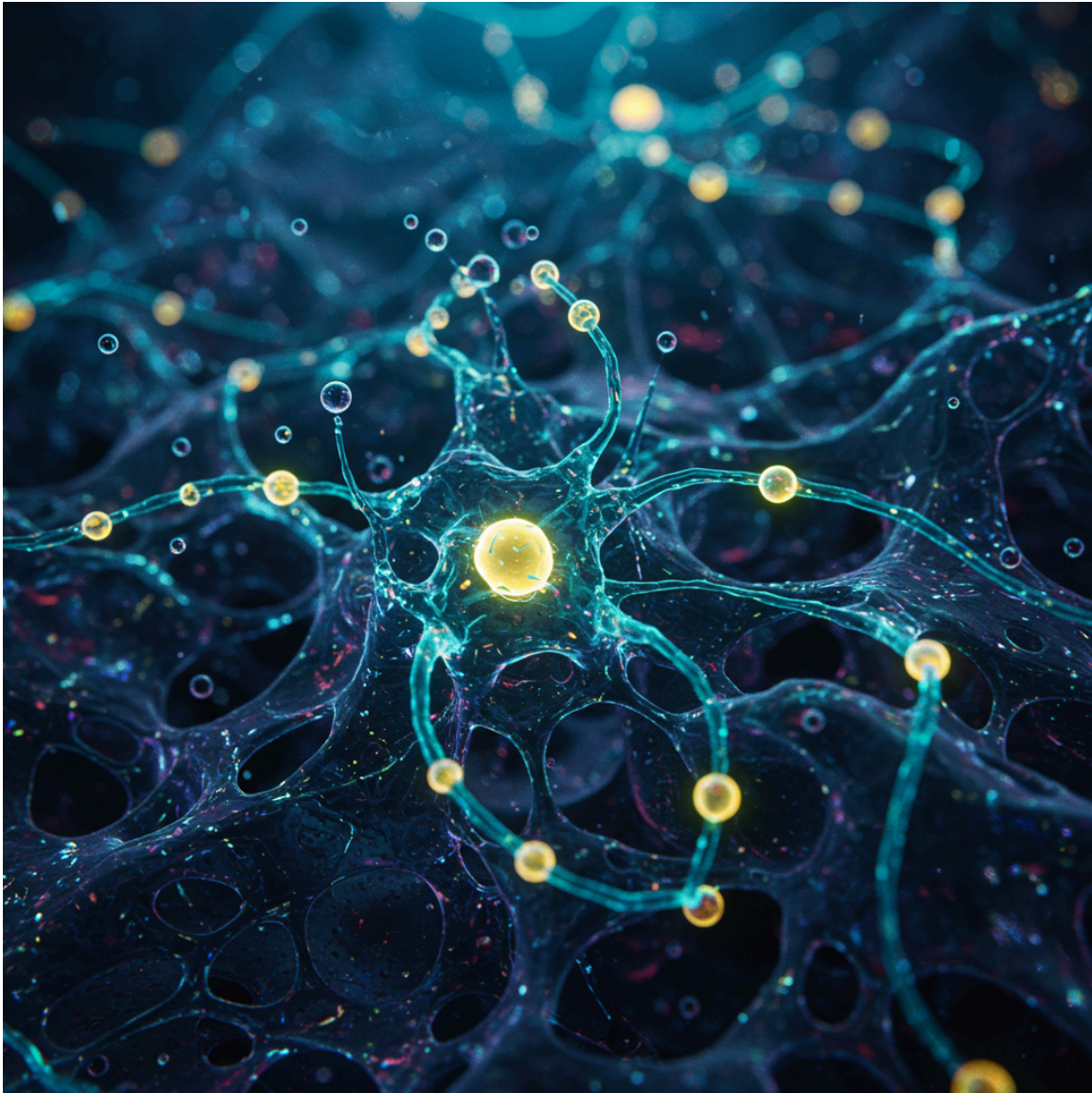




Assignment #1

PROJECT 1: EARLY PREDICTION OF SEPSIS

Simeon Schulz | ID: 305089317 | CSC-296S-01 | 2/24/26
Benjamin Church | ID: 302299231 | CSC-296S-01 | 2/24/26

Problem Statement

Sepsis is defined as life-threatening organ dysfunction caused by infection. In 2017, Sepsis globally affected 49 million people with approximately 11 million deaths and is the third leading cause of death in United States hospitals. While vigorous research has been made on sepsis, no single treatment has been discovered for curing sepsis. Early prediction of sepsis is crucial to providing appropriate medical treatment for patients.

In this project, Benjamin Church and Simeon Schulz explored (fully-connected) neural networks to make a binary (positive or negative) prediction of the onset of sepsis for each patient.

Methodology

Data Preparation

Based on one of the extra credits, it was determined the best approach by far was to perform undersampling on the dataset. Therefore, when reading in the dataset, priority was given to the minority of data, being sepsis patients. A for loop ran through both A and B datasets returning every entry of a patient who contracted sepsis, which came out to approximately 2900 entries. 3100 non sepsis patients were then added from dataset A, bringing the total to 6000 entries.

The data then needed to be prepared, the 5 extra columns dropped and the data needed to be flattened to 2D for FCNN models. However, we needed to determine what window size would be used, as some entries had less time entries than others. A test script was made determining the person in our dataset who had the least measurements on their record was 8 times total, so we went with window size 8 (meaning only the first 8 entries from each patient made it into the model). Global averages were also calculated and used to fill in NaN values in the case where local values didn't exist (an entire column was NaN). This data was then flattened, broken into x and y (x being all of the values over 8 time periods, and y being the final sepsis/non sepsis value recorded in the file), and put into train test split with randomization. At this point the data was now ready for training and testing models.

Additionally, for the LSTM a variant of this process was written that performs the same functionality but skipping the flatten() method at the end, allowing for x shape (10000, 8, 33) wherein we have 10000 patients, 8 time stamps per patient and 33 data values per time stamp.

FCNN Model Versions

More models can be found in the jupyter notebooks

Base Model

```
model = Sequential()
model.add(Dense(64, input_dim=x_train.shape[1], activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(16, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(8, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer=SGD(learning_rate=0.001, momentum=0.01), metrics=['accuracy'])
monitor = EarlyStopping(monitor='val_loss', min_delta=1e-3, patience=10, verbose=2, mode='min', restore_best_weights=True)
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), batch_size=64, callbacks=[monitor, checkpointer], verbose=1, epochs=10)
```

- 5 Layer (64, 32, 16, 8, 1)
- Dropout of 0.3 after each layer
- ReLu Activation Function
- SGD (lr = 0.001, momentum = 0.01)
- Early Stopping (min_delta = 1e-3, patience = 10)

Label	Precision	Recall	F1-Score	Support
Non-Sepsis	0.73	0.92	0.81	594
Sepsis	0.90	0.66	0.76	606

ROC AUC: 0.8735

Positives:

- Strong precision in sepsis (0.90)
- Strong recall in non-sepsis (0.92)

Drawbacks:

- Rather long training time (576 epochs)
- Model shows weakness in F1-Score sepsis label (0.76)

Model 1

Changes:

- Less layer
- Removed harsh dropout at every layer
- One dropout of 0.2 to prevent overfitting

```
model = Sequential()  
model.add(Dense(64, input_dim=x_train.shape[1], activation='relu'))  
model.add(Dropout(0.2))  
model.add(Dense(32, activation='relu'))  
model.add(Dense(16, activation='relu'))  
model.add(Dense(1, activation='sigmoid'))  
model.compile(loss='binary_crossentropy', optimizer=SGD(learning_rate=0.001, momentum=0.01), metrics=['accuracy'])  
monitor = EarlyStopping(monitor='val_loss', min_delta=1e-3, patience=10, verbose=2, mode='min', restore_best_weights=True)  
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), batch_size=64, callbacks=[monitor, checkpointer], verbose=1, epochs=10)
```

- 4 Layer (64, 32, 16, 1)
- Dropout of 0.2 after first layer
- ReLu Activation Function
- SGD (lr = 0.001, momentum = 0.01)
- Early Stopping (min_delta = 1e-3, patience = 10)
-

Label	Precision	Recall	F1-Score	Support
Non-Sepsis	0.77	0.87	0.82	594
Sepsis	0.85	0.75	0.80	606

ROC AUC: 0.8841

Positives:

- Strong F1-Score in sepsis label (0.80) compared to base model
- More balanced F1-Score (0.82 vs. 0.80)

Drawbacks:

- Rather long training time (312 epochs)

Model 2

Changes:

- Less neurons
- Leaky ReLu instead of normal ReLu

```
model = Sequential()  
model.add(Dense(32, input_dim=x_train.shape[1]))  
model.add(LeakyReLU(alpha=0.1))  
model.add(Dropout(0.2))  
  
model.add(Dense(16))  
model.add(LeakyReLU(alpha=0.1))  
  
model.add(Dense(8))  
model.add(LeakyReLU(alpha=0.1))  
  
model.add(Dense(1, activation='sigmoid'))  
  
model.compile(loss='binary_crossentropy', optimizer=SGD(learning_rate=0.001, momentum=0.01), metrics=['accuracy'])  
monitor = EarlyStopping(monitor='val_loss', min_delta=1e-3, patience=10, verbose=2, mode='min', restore_best_weights=True)  
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), batch_size=64, callbacks=[monitor, checkpointer], verbose=1, epochs=1)
```

- 4 Layer (32, 16, 8, 1)
- Dropout of 0.2 after first layer
- Leaky ReLu Activation Function (0.1)
- SGD (lr = 0.001, momentum = 0.01)
- Early Stopping (min_delta = 1e-3, patience = 10)

Label	Precision	Recall	F1-Score	Support
Non-Sepsis	0.75	0.85	0.80	594
Sepsis	0.84	0.73	0.78	606

ROC AUC: 0.8720

Positives:

- Less learnable parameters

Drawbacks:

- Overall weaker scores compared to model 1.
- Weakness in sepsis F1-Label (0.78)
- 449 epochs, long training time

Model 4 - Best Model

Changes:

- Learning rate scheduler
- Higher initial learning rate with bigger momentum
- Normal ReLu

```

checkpointer = ModelCheckpoint("model_less_layer_fcn-best_weights.keras", verbose=0, save_best_only=True)
lr_scheduler = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=5,
    verbose=1
)
best_val_loss = float('inf')
best_model = None

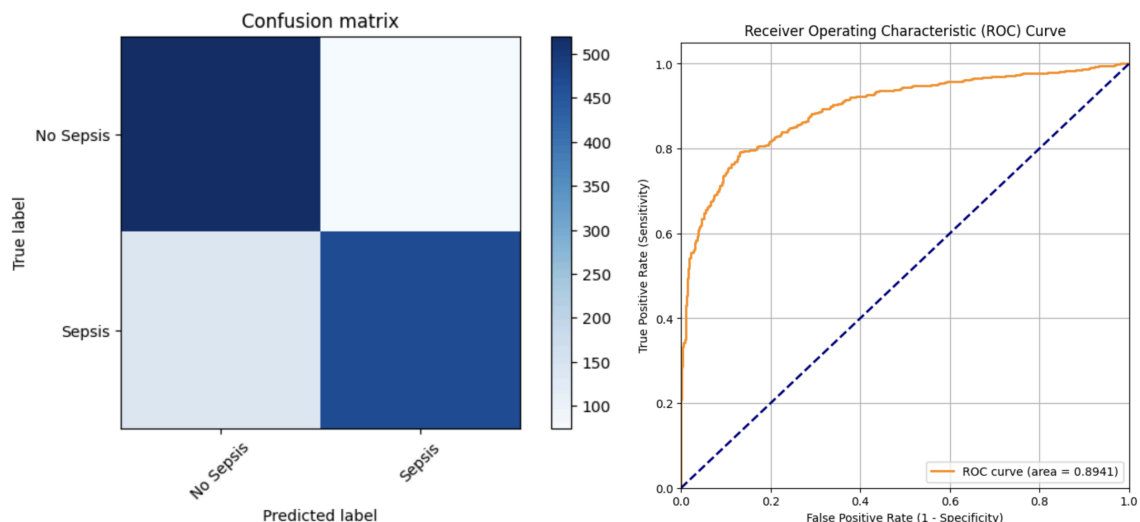
model = Sequential()
model.add(Dense(64, input_dim=x_train.shape[1], activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(32, activation='relu'))
model.add(Dense(16, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer=SGD(learning_rate=0.01, momentum=0.1), metrics=['accuracy'])
monitor = EarlyStopping(monitor='val_loss', min_delta=1e-3, patience=12, verbose=2, mode='min', restore_best_weights=True)
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), batch_size=64, callbacks=[monitor, checkpointer, lr_scheduler], verbose=0)

```

- 4 Layer (64, 32, 16, 1)
- Dropout of 0.2 after first layer
- ReLu Activation Function
- SGD (lr = 0.01, momentum 0.1)
- Early Stopping (min_delta = 1e-3, patience = 12)
- LR Scheduler (factor = 0.5, patience = 5)

Label	Precision	Recall	F1-Score	Support
Non-Sepsis	0.79	0.88	0.83	594
Sepsis	0.86	0.77	0.82	606



Positives:

- Higher overall scores (ROC AUC: 0.8941)
- Balanced and good F1-Score in each label (0.83, 0.82)
- Significantly shorter learning time (50 Epochs)

Drawbacks:

- Weak recall score for sepsis (0.77)

Model 5

Changes:

- Adam Optimizer
- Dropout (0.3) after each layer
- No learning rate scheduler

```
model = Sequential()
model.add(Dense(64, input_dim=x_train.shape[1], activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(16, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=0.001), metrics=['accuracy'])
monitor = EarlyStopping(monitor='val_loss', min_delta=1e-3, patience=8, verbose=2, mode='min', restore_best_weights=True)
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), batch_size=64, callbacks=[monitor, checkpointer], verbose=1, epochs=10)
```

- 4 Layer (64, 32, 16, 1)
- Dropout of 0.3 after each layer
- ReLu Activation Function
- Adam (lr = 0.001)
- Early Stopping (min_delta = 1e-3, patience = 8)

Label	Precision	Recall	F1-Score	Support
Non-Sepsis	0.79	0.87	0.83	594
Sepsis	0.86	0.77	0.81	606

ROC AUC: 0.8970

Positives:

- Balanced and good F1-Score in each label (0.83, 0.81)
- Significantly shorter learning time (13 Epochs)

Drawbacks:

- Tends to overfit a little

LSTM Model - Best Model

```
model = Sequential()
model.add(LSTM(128, input_shape=(WINDOW_SIZE, n_features), return_sequences=True))
model.add(Dropout(0.3))
model.add(LSTM(64, return_sequences=True))
model.add(Dropout(0.3))
model.add(LSTM(32, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(16, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
monitor = EarlyStopping(monitor='val_loss', min_delta=1e-3, patience=10, verbose=2, mode='min', restore_best_weights=True)
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), batch_size=64, callbacks=[monitor, checkpointer], verbose=1, epochs=10000)
```

- 3 LSTM Layer (128, 64, 32)
- 2 Dense Layer (16, 1)
- Dropout of 0.3 or 0.2 after each layer
- Relu Activation Function on only dense layer outside of output
- Adam (lr = 0.001)
- Early Stopping (min_delta = 1e-3, patience = 10)
-

Label	Precision	Recall	F1-Score	Support
Non-Sepsis	0.87	0.83	0.84	2000
Sepsis	0.88	0.88	0.88	2000

ROC AUC: 0.9116

Positives:

- Best performer
- Much less epochs

Drawbacks:

- Much slower training time (Nearly 5 seconds an epoch)
- Very accurate with non sepsis but struggled somewhat with sepsis

Experimental Results and Analysis

Given by an old project, a five layer deep (64,32,16,8,1) FCNN model was used to perform the first prediction task. However with a rather long training time of roughly 2 minutes (443 epochs) and insufficient results (F1-Score: Non-Sepsis = 0.81, Sepsis = 0.74), a slimmer model was needed. Thus Model 1 was introduced, which held a 4 layer deep architecture (64, 32, 16, 1). While the evaluation metrics showed better results (F1-Score: Non-Sepsis = 0.83, Sepsis = 0.81) the long training time of 385 epochs was still too slow. Using less neurons and leaky ReLu to prevent dead neurons for Model 2 did not seem to help with yielding better metrics (F1-Score: Non-Sepsis = 0.81, Sepsis = 0.78) nor fastening the training time (459 epochs). After various other architectural experiments, Model 4 was born, which yielded the best evaluation metrics (F1-Score: Non-Sepsis = 0.83, Sepsis = 0.82) and is going to be analyzed in detail.

A 4 layer deep architecture (64, 32, 16, 1) is complex enough to capture intricate data, while keeping the trainable parameters to a minimum. A dropout of 0.2 after the first layer, helps prevent overfitting during the training phase. The ReLu activation function brings a non-linearity part into the process for a better classification. The SGD (Stochastic Gradient Descent) optimizer with an initial learning rate of 0.1 and a momentum of 0.1 helps starting off strong and with big steps in the loss landscape. One can imagine it like a heavy bowling ball rolling through the loss landscape without getting stuck in small local loss valleys (due to the momentum of 0.1). The learning rate scheduler is what cut down the training time significantly. With a factor of 0.5 after 5 epochs without improvement in the validation loss, the learning rate is cut in half, allowing it to narrow down on the global loss minimum without skipping over it. The early stopping system with following parameters: min_delta = 1e-3, patience = 12 prevents overfitting by watching the improvement of the validation loss. The optimal performance of the model was found after 62 epochs.

Analyzing the evaluation metrics, the precision of 0.79 for non-sepsis cases shows that the model is accurate in predicting 79% of non-sepsis cases with 21% false positives. This score is ok. In sepsis cases, the model was accurate in 86% of all cases with 14% false positives. This shows strength in precision for the sepsis cases. With a recall of 88% in non-sepsis cases and 12 false negatives, the model misses less non-sepsis cases than sepsis cases. The recall of sepsis cases is 77% which means that the model misses 23% of sepsis cases. This is concerning, as not missing any sepsis cases is the most important metric for real world applications. The F1-Score of 83% for non-sepsis and 82% for sepsis shows a good balance in both classes. The support of 594 for non-sepsis and 606 for sepsis makes the classes balanced and the metrics comparable. The confusion matrix shows two clear blue boxes for making the right prediction while being slightly stronger in the no-sepsis case. The most mistakes were made in the “false positive” sector. The ROC Curve with an area of 0.8941 shows a distinctive learning ability of the model.

Later, it was determined that this data would be a good candidate for time series analysis, via an LSTM. Ideally, an LSTM would be able to utilize the time aspect of the data in order to better predict whether the patient may be showing signs over time of contracting sepsis (increasing levels of some values, decreasing of others, etc). In practice, this model started strong with a validation loss of only 0.38, but struggled to improve much from there, reaching a best loss of about 0.3. It had the best ROC AUC score of 0.9116, having extremely high accuracy on non sepsis patients, but struggled somewhat with non sepsis patients (based on the confusion matrix). It is my presumption that there may be a sweet spot in the window size that would help this model to perform better, striking a balance between filling in as few as possible of the shorter entries (max window sizes of 8,9, etc), while also giving a big enough window size such that some of the larger files (window size 20+) could still effectively be trained.

Task Division and Project Reflection

Benjamin: Data Preprocessing, EC 1, EC 4, Testing various model architectures

Simeon: Testing various model architectures, EC 3, Evaluation and Report

The initial challenge came with determining how to balance the data. After attempting to read in the first 6000 lines and looking at the ratios, it was discovered that only roughly 500 of the 6000 entries were sepsis patients, not nearly enough for the model to greatly understand the patterns correlated with sepsis. This was amended using a form of undersampling, by first reading in every sepsis patient we have available and getting roughly the same number of non sepsis patients afterwards. This meant that the model would be able to equally learn on sepsis and non sepsis patients.

Another challenge was to find out how to reduce the long training time for the models. For the first models, the training phase took around 2 minutes with more than 400 epochs. This seemed to be due to the small learning rate of 0.001 with a small momentum of 0.01 while using the SGD optimizer. However, introducing a learning rate scheduler which decreased an initial bigger learning gradually, while using a bigger momentum, helped locate the global minima in the loss landscape significantly faster. With 69 epochs, the training time was reduced to around 13 seconds. Furthermore, using the more sophisticated Adam optimizer also helped reduce the training time significantly.

As a Team we have learned that proper pre processing the data is equally as important as figuring out the optimal model architecture and hyperparameters. Only with careful preparation where turning the raw data into usable training/test data, the model can achieve its best outcome.

Additional Features

EC1: Undersampling

From the dataset, it was evident that there were significantly more entries for patients that did not contract sepsis than patients who did. Therefore, the working assumption is that the model may not have enough sepsis data to train off of. A test was conducted between using the first 6000 samples read in from the dataset, and a downsampling technique where all instances of sepsis patients were read in first (approx 2900), and the remaining 3100 entries were filled in with the first non sepsis patients found in dataset A (this was later shuffled). Both tests were conducted using the same model architecture.

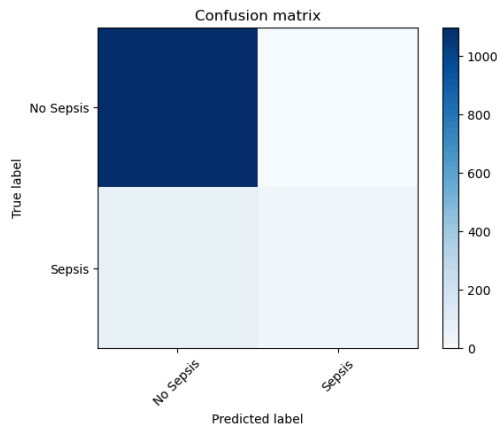


Figure 1: First 6000 Patients

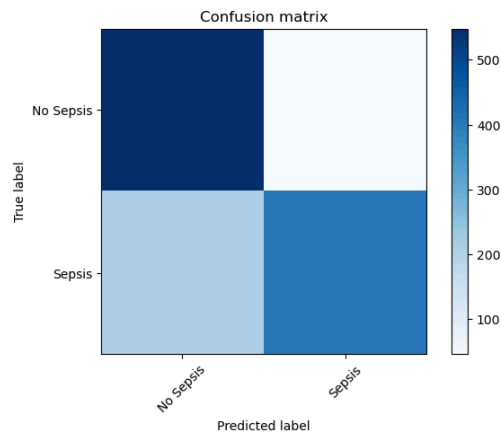


Figure 2: Undersampling

As can be seen from the confusion matrices, the undersampled version was seemingly unable to understand the pattern between sepsis and non sepsis, defaulting to only predicting non sepsis as it was the more frequent result. An additional test was conducted to find the ratio of sepsis and non sepsis patients from the first 6000 entries, coming out to 537 sepsis patients and 5463 non sepsis patients, leading to heavy bias. It can therefore be determined that some form of undersampling is necessary for this dataset in order to achieve accurate results.

EC3: Weight Variation in Training Samples

Predicting a sepsis correct without missing a case is the most important aspect in the sepsis prediction process. Since we used undersampling for our non-sepsis class, we have a balanced dataset which offers equal learning chances for both classes for the model. However we still noticed that especially the Recall score on sepsis cases was continuously lower than in the non-sepsis cases. Noticing that, we implemented class weights to punish errors on sepsis cases stronger. Thus the model learns to focus more on sepsis cases. After trying out different weights, the following weights yielded the best results:

- Non-sepsis: 1.0 (class weight remains the same)
- Sepsis: 1.8 (class weight is 1.8 as important as non-sepsis)

Comparing the evaluation metrics of the natural model to the same model with class weights applied shows that the class weights help achieve significantly more balanced results throughout precision, recall and F1-Score for both classes:

Without class weights:

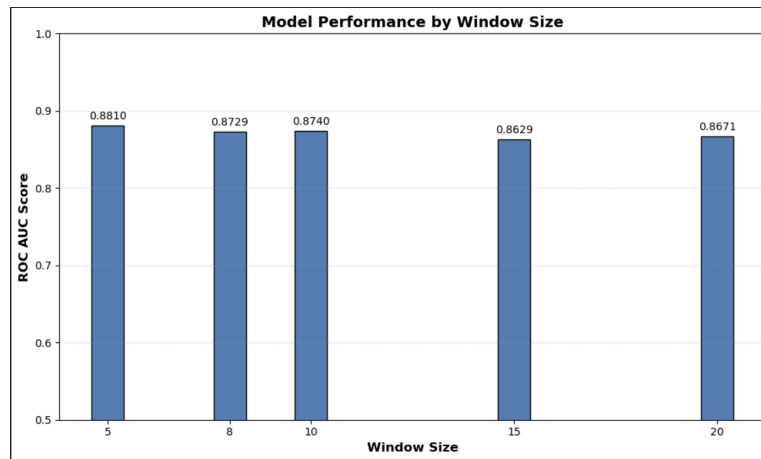
Label	Precision	Recall	F1-Score	Support
Non-Sepsis	0.79	0.88	0.83	594
Sepsis	0.86	0.77	0.82	606

With class weights:

Label	Precision	Recall	F1-Score	Support
Non-Sepsis	0.82	0.82	0.82	594
Sepsis	0.82	0.82	0.82	606

EC4: Comparison of Different Window Sizes

This data was broken into time stamps, where the patient was measured with the same statistics at different times, allowing us to decide how much or how little of the data we wish to include for our models. A test program was written to find the smallest window size of a patient included in our loaded in dataset, which came out to 8 (at least one patient in our dataset was only measured 8 times). Therefore, our base case was set to 8 for our models. Further evaluation was done later, with a new function allowing us to fill in empty rows with global averages, allowing the testing of window sizes 10, 15 and 20, as well as 8 and 5. Results were as follows



Note that each case here is using the same model architecture and is run 5 times to help with the gradient descent problem. With these results, it could be determined that sizes 15 and 20 perform significantly worse than 5 8 and 10, suggesting that filling in too much with global means may have a negative effect on the model's ability to learn from the data. Of these versions it's apparent that window size 5 performed the best.